

Technical Assessment

Forward Deployment Engineer

Role Forward Deployment Engineer	Time Budget 3-5 hours	Submission GitHub repository
---	--	---

Context

We are a B2B company. We make companies radically more efficient by building AI products. We move fast and we constantly ship.

As a Forward Deployment Engineer you prototype, integrate, and deliver working AI-powered systems directly at the client interface. This assessment is a compressed version of what that looks like in practice.

The Scenario

You are building a voice AI agent for an airline's customer service line, powered by ElevenLabs. The agent must handle inbound calls and assist customers across a range of requests.

To get started, you will write a basic system prompt; one that gives the agent sufficient instructions regarding how it should handle the user flow when it comes to handling calls made by clients who call the airline's telephone line. This is your baseline. The rest of the assessment is about making it self-correct.

Customer Scenarios to Support

Your system must be able to handle all of the following call types:

- Book the next available flight to a requested destination
- Find and book the cheapest available tickets within the following week
- Enquire whether pets are permitted on board and under what conditions
- Reschedule an existing booking to a different date or flight
- Enquire about baggage allowance; weight limits, cabin vs hold, excess fees
- Request a refund or cancellation for an existing booking
- Book a flight with a specific seat preference (window, aisle, extra legroom)
- Add an extra bag or special item (pram, sports equipment) to an existing booking
- Enquire about check-in times, gate information, or flight status
- Request assistance for a passenger with reduced mobility or special needs

What to Build

1: Knowledge Base

Create a knowledge base your agent can query during a call. It must cover at minimum: pet policy, baggage allowance rules and fees, special assistance options, check-in windows, and cancellation and refund policy. You define the content; make it realistic and internally consistent. You can use anything; a DB, a JSON, a custom API, whatever works best for you.

2: Flight Database

Create a fictional database of available flights across a single week. Include multiple destinations, departure times, seat classes, and pricing tiers. This is what your booking APIs will read from and write to.

3: Booking APIs and Webhooks

Build the backend API layer that your ElevenLabs agent will call via webhooks. At minimum, implement:

- Search available flights by destination, date, or price
- Book a flight (write to database, prevent double bookings)
- Cancel or reschedule a booking
- Add baggage or special items to an existing booking
- Retrieve an existing booking by reference

All operations must persist to a database of your choice. Bookings made by one simulated caller must be visible to the next.

4: The Autonomous Refinement Loop

This is the core of the assessment. Build a Python module that runs the following cycle autonomously, without any human intervention, until all scenarios pass or the iteration limit is reached:

1. Simulate a conversation. An LLM plays a customer with one of the scenarios above. Your ElevenLabs agent plays the airline assistant. The conversation runs via the ElevenLabs text-mode (Chat Mode) API; no audio required.
2. Evaluate the conversation. A separate LLM call scores the transcript against defined criteria: was the customer's request correctly understood, was the right API called with correct parameters, was the outcome confirmed clearly, was the call handled naturally end to end. Scores are 1–10 per criterion with specific failure quotes. Critically, the evaluator must also classify the root cause of each failure: is it a prompt issue (the agent misunderstood or was poorly instructed) or a code issue (an API returned wrong data, a webhook misfired, logic was incorrect).
3. Fix what is broken. If the root cause is a prompt issue, an LLM rewrites the system prompt targeting the identified failures. If the root cause is a code issue, an LLM generates a patch for the relevant module; the webhook handler, API logic, database query, or otherwise. Both types of fix can occur in the same iteration. Code patches are applied automatically and the affected service is reloaded before the next run.
4. Push the updated prompt to ElevenLabs via their REST API if the prompt changed.
5. Repeat from step 1 with the updated agent and updated code.

Termination The loop stops when all evaluation scores reach 8/10 or higher, or after 5 iterations. Both thresholds must be configurable constants.

On code patching

The system does not need to handle arbitrary refactors. Targeted, file-level patches to specific functions are sufficient. The evaluator should identify which file and which behaviour is at fault; the fixer acts on that signal.

5: Interface

Build a UI that lets someone observe the loop running in real time. It must show the live conversation feed, evaluation scores per iteration, a diff of what changed in the prompt, and an iteration history. No manual intervention should be possible or necessary; the UI is for observation only.

Deliverables

- A GitHub repository with all working code
- A README covering: how to run the project, APIs and tools used, tradeoffs made, and what you would improve next
- One recorded example run: the starting prompt, scores per iteration, and the final refined prompt
- A structured log file generated by each pipeline run, capturing: which scenario was tested, the evaluation scores per iteration, the root cause classification (prompt vs code), what was changed and why, and a summary of performance improvement from first to final iteration.

Evaluation Criteria

Area	What we are looking for	Weight
The loop works	Does the pipeline run autonomously, iterate, and terminate correctly? Does it correctly distinguish prompt failures from code failures and apply the right fix?	30%
Integration quality	Do the webhooks, flight database, and knowledge base work together reliably? No double bookings, no hallucinated data.	25%
Prompt engineering	Do the evaluation and refinement prompts produce consistent, structured, actionable output across multiple runs?	20%
UI clarity	Can someone watch the loop and understand exactly what is happening at each step?	15%
Communication	Does the README explain decisions honestly, including tradeoffs?	10%

Submission

Send your GitHub link within the agreed deadline. Be prepared for a walkthrough call where we will run the pipeline live, ask you to explain specific prompt decisions, adjust the pass threshold and re-run, and discuss what production readiness would require.

Note

AI coding assistants are permitted and expected. What matters is the quality of the result and your ability to explain every decision you made.